

(Task 1 of 20) In this tutorial, we will learn *more* about **mutable values**, illustrated with **vectors**.

0

(Task 2 of 20) What is the result of running this program?

Pseudo | 

Pseudo [Run ▶]

JavaScript

```
let x = vec[43, 54]   let x = [ 43, 54 ];
x[0] = x              x[0] = x;
print(x[1])           console.log(x[1]);
```

54

2

You predicted the output correctly 🎉🎉🎉

3

`x` is bound to a vector. `x[0] = x` makes the vector refer to itself. This is fine because a vector element can be any value, including itself. Besides, the vector is not copied, so the mutation finishes immediately.

Click [here](#) to run this program in the Stacker.

(Task 3 of 20) What is the result of running this program?

Pseudo | 

Pseudo [Run ▶]

Python

```
let x = vec[74, 82]   x = [ 74, 82 ]
let y = vec[x]         y = [ x ]
x[0] = y               x[0] = y
print(x[1])            print(x[1])
```

74

5

The answer is 82.

6

► Textual explanation

What is the result of running this program?

Pseudo | 

Pseudo [Run ▶]

Python

```
let a = vec[41, 92]   a = [ 41, 92 ]
let b = vec[a]         b = [ a ]
a[1] = b               a[1] = b
print(a[0])            print(a[0])
```

41

9

You predicted the output correctly 🎉🎉🎉

Pseudo | 

(Task 4 of 20) What is the result of running this program?

Pseudo [Run ▶]

JavaScript

```
let x = vec[52, 70, 61]
let y = x
x[0] = 93
print(y[0])
```

```
let x = [ 52, 70, 61 ];
let y = x;
x[0] = 93;
console.log(y[0]);
```

93

11

Please briefly explain why you think the answer is 93.

12

variable references pass through from y to x

13

You predicted the output correctly 🎉🎉🎉

14

The program creates a vector where the 0-th element is 52. and binds it to both x and y. The 0-th element of the vector is then replaced with 93. So, the result is 93.

Click [here](#) to run this program in the Stacker.

Pseudo | 

(Task 5 of 20) What is the result of running this program?

Pseudo [Run ▶]

Lispy

```
let x = vec[19, 73, 28]
let y = vec[x, x]
y[0][0] = 64
print(y[1])
```

```
(defvar x (mvec 19 73 28))
(defvar y (mvec x x))
(vec-set! (vec-ref y 0) 0 64)
(vec-ref y 1)
```

vec[64, 73, 28]

16

You predicted the output correctly 🎉🎉🎉

17

This program first creates a three-element vector and binds it to x. After that, the program creates a two-element vector. Both elements refer to the first vector. After that, the 0-th element of the three-element vector is replaced with 64. Finally, the three-element vector is printed.

Click [here](#) to run this program in the Stacker.

Click [here](#) to run this program in the Stacker.

Pseudo 

(Task 6 of 20) What is the result of running this program?

Pseudo 

Python

```
let x = vec[99, 83]
fun f(y):
  x[0] = 34
  return y
end
print(f(x))
```

```
x = [ 99, 83 ]
def f(y):
  x[0] = 34
  return y
print(f(x))
```

vec[34, 83]

19

You predicted the output correctly 🎉🎉🎉

20

The program creates a vector and binds it to `x`. The function call `f(x)` binds `y` to the same vector. The function replaces the 0-th element of the vector with 34 and then returns the vector. So, the final result is the vector, which is printed as `vec[34, 83]`.

Click [here](#) to run this program in the Stacker.

Pseudo 

(Task 7 of 20) What is the result of running this program?

Pseudo 

Lispy

```
let x = vec[53]
let v = vec[72, x]
x[0] = 72
print(v)
```

```
(defvar x (mvec 53))
(defvar v (mvec 72 x))
(vec-set! x 0 72)
v
```

vec[53, vec[72]]

22

Please briefly explain why you think the answer is `vec[53, vec[72]]`.

23

it gets mutated correctly

24

The answer is `vec[72, vec[72]]`.

25

▼ Textual explanation

`x` is bound to a one-element vector. The only element of this vector is 53. `v` is bound to a two-element vector. The elements of this vector are 72 and the one-element vector. The subsequent `x[0] = 72` mutates the one-element vector. Finally, the value of `v` is printed.

Click [here](#) to run this program in the Stacker.

What is the result of running this program?

Pseudo |  27

Pseudo [Run 

JavaScript

```
let m = vec[0]
let v = vec[52, m, 53]
m[0] = 71
print(v)
```

```
let m = [ 0 ];
let v = [ 52, m, 53 ];
m[0] = 71;
console.log(v);
```

vec[52, 71, 53]

27

You predicted the output correctly 🎉🎉🎉

28

(Task 8 of 20) What is the result of running this program?

Pseudo |  30

Pseudo [Run 

Python

```
let x = vec[62]
let y = x
y[0] = 34
print(x)
```

```
x = [ 62 ]
y = x
y[0] = 34
print(x)
```

vec[34]

30

You predicted the output correctly 🎉🎉🎉

31

x is first bound to a vector that refers 62. y is bound to the same vector. The vector mutation replaces 62 with 34. So, eventually, the vector becomes vec[34].

Click [here](#) to run this program in the Stacker.

(Task 9 of 20) What is the result of running this program?

Pseudo |  32

Pseudo [Run 

Scala 3

```
let x = vec[10, 48, 95]
fun f(y):
  y[0] = 32
  return y[0]
end
let z = f(x)
print(x[0])
```

```
val x = Buffer(10, 48, 95)
def f(y : Int) =
  y(0) = 32
  y(0)
val z = f(x)
println(x(0))
```

32

33

You predicted the output correctly 🎉🎉🎉

34

The program first creates a three-element vector, where the 0-th element is 10. After that, the program defines a function `f`. The function call `f(x)` replaces the 0-th vector element with 32. After that, the vector is printed. So the result is 32.

Click [here](#) to run this program in the Stacker.

(Task 10 of 20) What did you learn about vectors from these programs?

35

The values get passed through even after assigning them to a variable

36

(Task 11 of 20) A vector can be referred to by more than one variable and even by other vectors (including itself). Referring to a vector does not create a copy of the vector; rather, they share the same vector. Specifically

37

- Binding a vector to a new variable does not create a copy of that vector.
- Vectors that are passed to a function in a function call do not get copied.
- Creating new vectors that refer to existing ones does not create new copies of the existing vectors.

The references share the same vector. That is, vectors can be **aliased**.

Any feedback regarding these statements? Feel free to skip this question.

38

(You skipped the question.)

39

(Task 12 of 20) Now please scroll back and select 1-3 programs that make the above points.

40

You don't need to select *all* such programs.

(You selected 1 programs)

41

Okay. How does this program (26) support the point?

42

`v` references the original `m` so any alteration there is made to

43

(Task 13 of 20) Reconsider these two programs that you might have seen. The only difference is in `let y = ____`.

Pseudo [Run 

Scala 3

```
let x = vec[52, 96]    val x = Buffer(52, 96)
let y = x              val y = x
x[0] = 34              x(0) = 34
print(y)              println(y)
```

Pseudo [Run 

Scala 3

```
let x = vec[52, 96]    val x = Buffer(52, 96)
let y = vec[52, 96]    val y = Buffer(52, 96)
x[0] = 34              x(0) = 34
print(y)              println(y)
```

It is tempting to describe the variables as

- `x` is bound to `vec[52, 96]`
- `y` is bound to `vec[52, 96]`

This description does not help us to understand the program because it can't explain why `x[0] = 34` mutates `y` in one case, and does not in the other.

What is a better way to describe the bindings that help solve this problem?

the variable `y` references the content of `x`

45

We can say, in the first program

46

- `x` is bound to `@100`
- `y` is bound to `@100`

where

- `@100` is `vec[52, 96]`

While for the other program

- `x` is bound to `@100`
- `y` is bound to `@200`

where

- `@100` is `vec[52, 96]`
- `@200` is `vec[52, 96]`

(Task 14 of 20) In SMoL, each vector has its own unique **heap address** (e.g., `@100` and

47

@200). The mapping from addresses to vectors is called the **heap**.

(**Note:** we use @ddd (e.g., @123, @200, and @100) to represent heap addresses. Heap addresses are *random*. The numbers don't mean anything.)

Any feedback regarding these statements? Feel free to skip this question.

48

(You skipped the question.)

49

(Task 15 of 20) Which choice best describes the status of the heap at the end of the following program?

Pseudo | 

Pseudo [Run 

Python

```
let x = 3          x = 3
let v = vec[1, 2, x]  v = [ 1, 2, x ]
```

51

- A. @1 = #(1 2 3)
- B. @1 = #(1 2 x)
- C. There is nothing in the heap.

@1 = #(1 2 x)

52

The answer is @1 = #(1 2 3).

53

C is wrong because the vec[1, 2, x] creates a vector. Every vector is stored on the heap.

B is wrong because vectors refer values, while x is not a value.

(Task 16 of 20) Which choice best describes the status of the heap at the end of the following program?

Pseudo | 

Pseudo [Run 

Python

```
let mv = vec[3]      mv = [ 3 ]
let mv2 = vec[mv, mv]  mv2 = [ mv, mv ]
mv2[0][0] = 42       mv2[0][0] = 42
```

55

- A. @100 = #(3); @200 = #(@100 @100)
- B. @100 = #(3); @200 = #(@300 @100); @300 = #(42)
- C. @100 = #(3); @200 = #(#(42) #(3))

- D.** `@100 = #(42); @200 = #(@100 @100)`
E. `@100 = #(42); @200 = #(#(42) #(42))`
F. There is nothing in the heap.

`@100 = #(3); @200 = #(#(42) #(3))`

56

The answer is `@100 = #(42); @200 = #(@100 @100)`.

57

E and **C** are wrong. Vectors refer values. `vec[42]` and `vec[3]` are not values, although some vector values are printed like them. So, `@200 = #(#(42) #(42))` and `@200 = #(#(42) #(3))` can not be valid.

`vec[3]` creates a 1-element vector, `@100`. The only element is 3. `vec[mv, mv]` creates a 2-element vector, `@200`. Both elements of `@200` are `@100`. No more vectors are created, which means **B** must be wrong. So, then, the correct answer must be **A** or **D**.

However, the subsequent mutation changes `@100` (the first element of `@200`). The 0-th element of `@100` is mutated to 42. So, **D** is the correct answer.

Pseudo | 

(Task 17 of 20) The following program defines two variables but creates nothing on the heap.

Pseudo [Run]

JavaScript

```
let x = 2    let x = 2;
let y = 3    let y = 3;
print(x)    console.log(x);
print(y)    console.log(y);
```

The following program defines no variables but creates two things on the heap.

Pseudo [Run]

JavaScript

```
print(vec[1, vec[2, 3]])    console.log([ 1, [ 2, 3 ] ]);
```

What did you learn from this pair of programs?

vectors are heap-bound

59

(Task 18 of 20)

60

- Creating a vector does not inherently create a binding.
- Creating a binding does not necessarily alter the heap.

Any feedback regarding these statements? Feel free to skip this question.

61

(You skipped the question.)

62

(Task 19 of 20) Here is a program that confused many students

Pseudo | 

Pseudo [Run 	Lispy <pre> let v = vec[1, 2, 3, 4] (defvar v (mvec 1 2 3 4)) let vv = vec[v, v] (defvar vv (mvec v v)) vv[1][0] = 100 (vec-set! (vec-ref vv 1) 0 100) print(vv) vv </pre>
--	--

Please

1. Run this program in the stacker by clicking one of the [Run  buttons above;
2. The stacker would show how this program produces its result(s);
3. Keep clicking ▶▶ Next until you reach a configuration that you find particularly helpful;
4. Click  Share This Configuration to get a link to your configuration;
5. Submit your link below;

https://smol-tutor.xyz/stacker/?syntax=Pseudo&inputSyntax=Lispy&randomSeed=smol-tutor&hole=%E2%80%A2&nNext=1&program=%0A%28defvar+v+%28mvec+1+2+3+4%29%29%0A%28defvar+vv+%28mvec+v+v%29%29%0A%28vec-set%21+%28vec-ref+vv+1%29+0+100%29%0Avv%0A&readOnlyMode=

64

(Task 20 of 20) Please write a couple of sentences to explain how your configuration explains the result(s) of the program.

65

this works because it passes without binding the heap

66

Let's review what we have learned in this tutorial.

67

A vector can be referred to by more than one variable and even by other vectors (including itself). Referring to a vector does not create a copy of the vector; rather, they share the same vector. Specifically

- Binding a vector to a new variable does not create a copy of that vector.
- Vectors that are passed to a function in a function call do not get copied.
- Creating new vectors that refer to existing ones does not create new copies of the existing vectors.

The references share the same vector. That is, vectors can be **aliased**.

In SMoL, each vector has its own unique **heap address** (e.g., @100 and @200). The mapping from addresses to vectors is called the **heap**.

(**Note:** we use @ddd (e.g., @123, @200, and @100) to represent heap addresses. Heap addresses are *random*. The numbers don't mean anything.)

- Creating a vector does not inherently create a binding.
- Creating a binding does not necessarily alter the heap.

You have finished this tutorial 🎉🎉🎉

Please [print](#) the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1790871176957